



Sprint 6: Datos listos para modelar

Una introducción práctica al ecosistema de datos en Python: desde la comprensión del rol del Data Scientist hasta la exploración visual de conjuntos de datos reales. En este sprint aprenderás a cargar, inspeccionar y visualizar datos con las herramientas más utilizadas en la industria.

S6.1 · DATOS, PANDAS Y VISUALIZACIÓN EXPLORATORIA

Contenidos del Sprint

Este sprint cubre cinco bloques fundamentales que te llevarán desde el rol profesional del Data Scientist hasta la elección del gráfico más adecuado para cada tipo de análisis.



Rol del Data Scientist

Qué hace, qué habilidades necesita y cómo encaja en un equipo de datos.



Tipos de dato

Estructurado, no estructurado y series temporales.



pandas

Carga e inspección de datos con la librería más popular de Python.



Visualización exploratoria

matplotlib, seaborn y plotly para explorar y comunicar datos.



¿Qué gráfico usar?

Criterios prácticos para elegir la visualización correcta.

BLOQUE 1

El Rol del Data Scientist

El Data Scientist es el profesional que extrae conocimiento y valor a partir de los datos. Su trabajo combina estadística, programación y comprensión del negocio para responder preguntas críticas y guiar decisiones estratégicas.



¿Qué hace un Data Scientist?

El Data Scientist actúa como puente entre los datos brutos y las decisiones de negocio. Sus responsabilidades abarcan todo el ciclo de vida del dato, desde la recolección hasta la comunicación de resultados.



Explorar y limpiar

Identificar problemas en los datos: valores nulos, outliers, inconsistencias y formatos incorrectos antes de cualquier análisis.



Modelar y predecir

Aplicar algoritmos de machine learning y estadística para construir modelos que respondan preguntas concretas del negocio.



Comunicar resultados

Traducir hallazgos técnicos a un lenguaje comprensible para equipos no técnicos, usando visualizaciones y narrativa clara.



Diseñar pipelines

Construir flujos de trabajo reproducibles y escalables para que los análisis puedan ejecutarse de forma continua y automatizada.

Tipos de Dato

¿Por qué importa el tipo de dato?

El tipo de dato determina qué herramientas, modelos y visualizaciones son apropiados. Confundir un dato estructurado con uno no estructurado puede llevar a análisis incorrectos o modelos que no aprenden correctamente.

Clasificar bien los datos es el primer paso para elegir la estrategia de análisis adecuada.

Dato estructurado

Organizado en filas y columnas. Fácilmente almacenable en tablas SQL o DataFrames. Ejemplo: registros de ventas, datos de clientes.

Dato no estructurado

Sin formato predefinido. Incluye texto libre, imágenes, audio y vídeo. Requiere técnicas especializadas como NLP o visión por computador.

Series temporales

Datos indexados en el tiempo con orden cronológico relevante. Ejemplo: precios de acciones, temperatura horaria, tráfico web diario.

Comparativa de Tipos de Dato

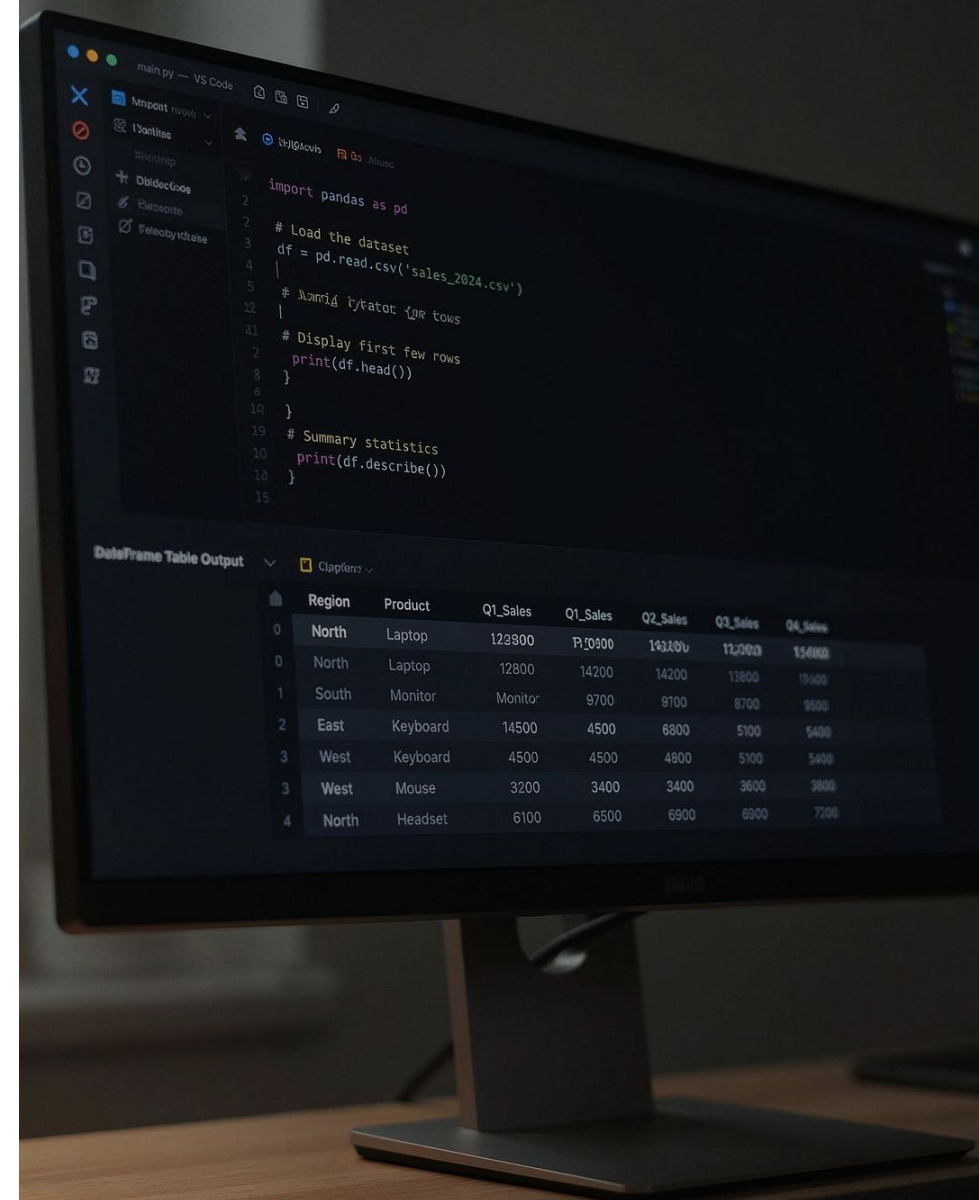
Cada tipo de dato tiene características propias que condicionan las técnicas de análisis y los modelos aplicables. Esta tabla resume las diferencias clave.

Característica	Estructurado	No estructurado	Series temporales
Formato	Tablas, CSV, SQL	Texto, imagen, audio	Tabla indexada por tiempo
Herramientas	pandas, SQL	NLP, CNN, OCR	pandas, statsmodels
Volumen típico	Medio	Alto	Medio-alto
Ejemplo	Tabla de clientes	Reseñas de productos	Precio de acciones

BLOQUE 3

pandas: Carga e Inspección de Datos

pandas es la librería de referencia para el análisis de datos tabulares en Python. Su estructura principal, el **DataFrame**, permite manipular, filtrar y transformar datos de forma intuitiva y eficiente, con una sintaxis limpia y muy expresiva.



Primeros Pasos con pandas

Cargar e inspeccionar un dataset son las primeras acciones en cualquier proyecto de datos. pandas ofrece funciones específicas para cada etapa del proceso de exploración inicial.

Carga de datos

```
import pandas as pd

# Cargar desde CSV
df = pd.read_csv("datos.csv")

# Cargar desde Excel
df = pd.read_excel("datos.xlsx")

# Cargar desde JSON
df = pd.read_json("datos.json")
```

Inspección básica

```
# Primeras filas del DataFrame
df.head()

# Dimensiones (filas, columnas)
df.shape

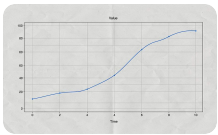
# Tipos de columnas y nulos
df.info()

# Estadísticas descriptivas
df.describe()
```

📌 Una buena inspección inicial te ahorrará horas de depuración más adelante. Siempre ejecuta `df.info()` y `df.describe()` antes de cualquier análisis.

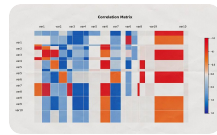
Visualización Exploratoria

La visualización exploratoria (EDA) permite entender la distribución, relaciones y anomalías en los datos **antes** de construir cualquier modelo. Python ofrece tres librerías complementarias que cubren desde gráficos estáticos hasta dashboards interactivos.



matplotlib

La librería base de visualización en Python. Ofrece control total sobre cada elemento del gráfico: ejes, colores, fuentes y estilos. Ideal para publicaciones científicas y gráficos muy personalizados.



seaborn

Construida sobre matplotlib, seaborn simplifica la creación de gráficos estadísticos atractivos. Destaca en visualizaciones de distribuciones, correlaciones y comparaciones entre grupos con muy pocas líneas de código.



plotly

Genera gráficos interactivos listos para web y dashboards. Permite hacer zoom, filtrar y explorar los datos directamente en el navegador. Perfecto para presentaciones y aplicaciones con Dash o Streamlit.

Comparativa de Librerías de Visualización

Criterio	matplotlib	seaborn	plotly
Curva de aprendizaje	Alta	Media	Media
Interactividad	No	No	Sí
Gráficos estadísticos	Básico	Excelente	Bueno
Personalización	Total	Alta	Alta
Uso recomendado	Publicaciones	EDA estadístico	Dashboards

¿Con cuál empezar?

Para exploración del día a día, **seaborn** es la opción más productiva: gráficos elegantes con poco código. Aprende **matplotlib** para personalizaciones avanzadas y **plotly** cuando necesites interactividad o presentaciones para clientes.

📌 Las tres librerías son complementarias. En un proyecto real se usan las tres en distintas etapas.

¿Qué Gráfico Usar y Cuándo?

Elegir el gráfico incorrecto puede distorsionar el mensaje y confundir a la audiencia. La elección depende del **tipo de variable** que quieres analizar y la **pregunta** que intentas responder.

DISTRIBUCIÓN



Histograma

COMPARACIÓN



Barras,
Columnas Agrupadas

RELACIÓN



Scatter Plot,
Correlation Heatmap

EVOLUCIÓN



Líneas
Área

COMPOSICIÓN



Pie Chart
Stacked Bars

EXPLORACIÓN



Pairplot
Parallel Coordinates

Guía Rápida: Tipo de Pregunta → Gráfico



¿Cómo se distribuye una variable?

Usa un **histograma** para ver frecuencias, un **boxplot** para detectar outliers o un **violinplot** para ver la distribución completa con densidad.



¿Cómo se comparan categorías?

Un **gráfico de barras** es la opción más clara. Si tienes muchas categorías, considera un gráfico de barras horizontal para mejorar la legibilidad.



¿Existe relación entre dos variables?

El **scatter plot** muestra correlaciones directamente. Complementa con una línea de tendencia o usa un **heatmap** para múltiples variables a la vez.



¿Cómo evoluciona algo en el tiempo?

El **gráfico de líneas** es el estándar para series temporales. El **gráfico de área** añade énfasis en el volumen acumulado a lo largo del tiempo.

Resumen: Lo Esencial del Sprint 6



Data Scientist

Combina estadística, código y visión de negocio para extraer valor de los datos.



Tipos de dato

Estructurado, no estructurado y series temporales. El tipo condiciona las herramientas.



pandas

`read_csv()`, `head()`, `info()` y `describe()` como punto de partida siempre.



Visualización

matplotlib para control, seaborn para EDA estadístico, plotly para interactividad.



Elegir gráfico

La pregunta que haces determina el gráfico que usas. Distribución, comparación, relación, evolución.